

Z-80
RELOCATING
MACRO
ASSEMBLER
USER'S MANUAL



COPYRIGHT 1976 BY
TECHNICAL DESIGN LABS INC.

TDL Z80 Relocating Assembler
User's Manual

Revision 1.2
December 9, 1976

Written by Neil J. Colvin

Copyright 1976 by Technical Design Labs, Inc.

Chapter 1

Introduction

The TDL Z80 Relocating Assembler is the symbolic assembly program for the Z80. It is a two-pass assembler (requiring the source program to be read twice to complete the assembly process) designed to run under the TDL system monitor. It is therefore device independent, allowing complete user flexibility in the selection of standard input and output device options.

The assembler performs many functions, making machine language programming easier, faster, and more efficient. Basically, the assembler processes the Z80 programmer's source program statements by translating mnemonic operation codes to the binary codes needed in machine instructions, relating symbols to numeric values, assigning relocatable or absolute memory addresses for program instructions and data, and preparing an output listing of the program which includes any errors encountered during the assembly.

The TDL Z80 Assembler also contains a powerful macro capability which allows the programmer to create new language elements, thus expanding and adapting the assembler to perform specialized functions for each programming job.

Statements

Assembler programs are usually prepared on a terminal, with the aid of a text editing program. A program consists of a sequence of statements in the assembly language. Each statement is normally written on one line, and terminated by a carriage return/line feed sequence. TDL assembler statements are free-format. This means that the various statement elements are not placed at a specific numbered column on the line.

There are four elements in an assembler statement (three of which are optional), separated from each other by specific characters. These elements are identified by their order of appearance in the statement, and by the separating (or delimiting) character which follows or precedes the elements.

Statements are written in the general form:

```
label: operator    operand,operand ;comment <CR-LF>
```

The assembler converts statements written in this form into the binary machine instructions.

Instruction Formats

The Z80 uses a variable length instruction format. A given machine instruction may be one, two, three, or four bytes long depending on the specific machine code and on the addressing mode specified. The TDL assembler automatically produces the correct number of machine code bytes for the particular operation specified. Appendix A specifies the various machine code mnemonics accepted by the assembler and the format of the operands required.

Statement Format

As previously described, assembler statements consist of a combination of a label, an operator, one or more operands, and a comment; the particular combination depends on the statement usage and operator requirements.

The assembler interprets and processes these statements, generating one or more binary instructions or data bytes, or performing some assembly control process. A statement must contain at least one of these elements, and may contain all four. Some statements have no operands, while others may have many.

Statement labels, operators, and operands may be represented numerically or symbolically. The assembler interprets all symbols and replaces them with a numeric (binary) value.

Symbols

The programmer may create symbols to use as statement labels, as operators, and as operands. A symbol may consist of any combination of from one to six characters from the following set:

The 26 letters: A-Z

Ten digits: 0-9

Three special characters:

 \$ (Dollar Sign)

 % (Percent)

 . (Period)

These characters constitute the Radix-40 character set (so named because it contains only 40 characters). Any statement character which is not in the Radix-40 set is treated as a symbol delimiter when encountered by the assembler.

The first character of a symbol must not be numeric. Symbols may also not contain embedded spaces. A symbol may contain more than six characters, but only the first six are used by the assembler.

The TDL assembler will accept programs written using

both upper and lower case letters and symbols. Lower case letters are treated as upper case in symbols. Additional special characters and lower case letters elsewhere are taken unchanged.

Labels

A label is the symbolic name created by the programmer to identify a statement. If present, the label is written as the first item in a statement, and is terminated by a colon (:). A statement may contain more than one label, in which case all identify the same statement. Each label must be followed by a colon, however. A statement may consist of just a label (or labels), in which case the label(s) identifies the following statement.

When a symbol is used as a label, it specifies a symbolic address. Such symbols are said to be defined (have a value). A defined symbol can reference an instruction or data byte at any point in the program.

A label can be defined with only one value. If an attempt is made to redefine a label with a different value, the second value is ignored, and an error is indicated.

The following are legal labels:

```
$SUM:  
ABC:  
B123:  
WHERE%:
```

The following are illegal:

```
30QRT:      (First character must not be a digit)  
AB CD:      (Cannot contain embedded space)
```

If too many characters are used in a label, only the first six are used. For example the label ZYXWVUTSR: is recognized by the assembler to be the same as ZYXWVUABC:.

Operators

An operator may be one of the mnemonic machine instruction codes, a pseudo-operation code which directs the assembly process, or a user defined code (either pseudo-op or macro): The assembler pseudo-op codes are described in Chapter 3 and summarized in Appendix B.

The operator element of a statement is terminated by any character not in the Radix-40 set (usually a space or a tab). If a statement has no label, the operator must appear first in the statement.

A symbol used as an operator must be predefined by the assembler or the programmer before its first appearance as an operator in a statement.

Operands

Operands are usually the symbolic addresses of the data to be accessed when an instruction is executed, the names of processor registers to be used in the operation, or the input data or arguments to a pseudo-op or macro instruction. In each case, the precise interpretation of the operand(s) is dependent on the specific statement operator being processed. Operands are separated by commas, and are terminated by a semicolon (;) or a carriage return/line feed.

Symbols used as operands must have a valued predefined by the assembler or defined by the programmer. These may be symbolic references to previously defined labels where the arguments used by this instruction are to be found, or the symbols may represent constant values or character strings.

Comments

The programmer may add a comment to a statement by preceding it with a semicolon (;). Comments are ignored by the assembler but are useful for documentation and later program debugging. The comment is terminated by the carriage return/line feed at the end of the statement. In certain cases (eg. conditional assembly and macro definitions), the use of the left and right square brackets ([]) should be avoided in a comment as it could affect the assembly process.

An assembler statement may consist of just a comment, but each such statement must begin with a semicolon.

Statement Processing

The assembler maintains several internal symbol tables for recording the names and values of symbols used during the assembly. These tables are:

1. Macro Table - This table contains all macros. It is initially empty, and grows as the programmer defines macros.
2. Op-Code Table - This table contains all of the machine operation mnemonics (op-codes), the assembler pseudo-operations (pseudo-ops), and user defined operators (.OPSYNs). It initially contains the basic op-codes and pseudo-ops, and grows as the programmer provides additional definitions.
3. Symbol Table - This table contains all programmer-defined symbols other than those described above. It initially contains the standard register names, and then grows as new symbols are defined.

Internally, all of these tables occupy the same space, so that all of the available space can be used as required.

Order of Symbol Evaluation

The following table shows the order in which the assembler searches the tables for a symbol appearing in each of the statement fields:

Label Field:

1. Symbol followed by a colon. If no colon is found, no label is present.

Operator Field:

1. Macro
2. Machine operator
3. Assembler operator
4. Symbol

Operand Field:

1. Number
2. Macro
3. Symbol
4. Machine operator

Because of the different table searching orders for each field, the same symbol could be used as a label, an operator, and a macro, with no ambiguity.

Programmer-Defined Symbols

There are two type of programmer-defined symbols: labels and assignments. As previously described, labels are generated by entering a symbol followed by a colon (e.g. LABEL:). Symbols used as labels cannot be redefined with a different value once they have been defined. The value of a label is the value of the location counter at the time the label is defined.

Assignments are used to represent, symbolically, numbers, bit patterns, or character strings. Assignments simplify the program development task by allowing a single source program modification (the assignment statement) to change all uses of that number or bit pattern throughout the program. Symbols given values in an assignment statement may have new values assigned in subsequent statements. The current value of an assigned symbol is the last one given to it.

A symbol may be entered into the symbol table with its assigned value by using a direct assignment statement of the form:

symbol = value {; or CR-LF}

where the value may be any valid numeric value or expression.

The value assigned to the symbol may subsequently be changed by another direct assignment statement.

The following are valid assignment statements:

```
VALUE1 = 23
SIZE = 4*36
ZETA = SIZE
```

If it is desired to fix the value assigned to a symbol so that it cannot subsequently be redefined, the direct fixed assignment statement should be used. This statement is the same as the direct assignment statement except that the symbol is followed by two equal signs instead of one. For example:

```
FIXED == 46
NEWVAL == SIZE
```

Assembly-Time Assignments

It is often desirable to defer the assignment of a value to a symbol until the assembly is actually underway (i.e. not specify the value as part of the source program). This is especially useful in setting program origin, buffer sizes, and in specifying parameter values which will be used to control conditional assembly pseudo-ops.

The TDL Assembler provides the ability to specify symbols with values to be determined at assembly time, and the mechanism by which the values may be interactively defined. To specify an assembly-time assignment, the following format is used:

symbol =\ [dtextd]

where the dtextd in brackets indicates the optional specification of a message to be output on the console device at assembly time before requesting the symbol's value. The d represents a text delimiter, and may be any character (other than a space or tab) which is not contained in the text itself. The text may contain carriage return/line feed sequences, which would result in a multi-line message on the console.

After the optional message is output on the console, a colon (:) is output to indicate that the assembler is waiting for the desired value to be entered. The value which is to be assigned to the symbol is then input on the console device and the assembly continues with the symbol having the specified value. This interaction only occurs

during the first assembly pass. The symbol's value remains unchanged during subsequent passes.

Only numeric values may be entered through the console in this fashion. The number which is input must conform to the same rules as any other number used in the assembly source program, and may be followed by an optional radix modifier (see the section on Numbers below). The number is assumed to be decimal unless followed by a radix modifier.

The value being input is not processed until a carriage return is entered. Any mistyped character may be deleted by the use of the DELETE (or RUBOUT) key (which will echo the deleted character), and the entire number may be deleted by entering CTL-U (simultaneous use of the CTRL and the U key). Any character which is input but is not valid as part of a number will not be echoed and will be ignored.

The following are examples of assembly-time assignment statements:

```
BUFSIZ =\ "BUFFER SIZE (50 TO 500 CHARACTERS)"  
DISK =\ "VERSION (0-PAPER TAPE 1-DISK)"
```

Assembly-time assignment statements are similar to direct fixed assignments (==) in not allowing the symbol to be redefined elsewhere in the program.

Local and Global Symbols

When assembling a large program, it is sometimes difficult to keep track of the symbols used for local data references and branching. To facilitate modular programming, the TDL assembler provides for both global and local symbols within a single program. All symbols which start with two periods are defined as being local, and all other symbols are global. For example, the following are valid local symbols:

```
..ABCD:  
..1234:  
...:
```

A particular occurrence of a local symbol is only defined within the boundaries of its enclosing global symbols. For example, in the following sequence of label definitions, the symbol ..SYM1 is only defined (and can only be referenced) within the program between the definition of GLOB1 and GLOB2:

```
GLOB1: ...  
..SYM1: ...  
GLOB2: ...  
...
```

This localization of symbol definitions allows the same symbol to be used unambiguously more than once in the program. It also simplifies program understandability by immediately differentiating between local and global symbols.

In addition to labels, any other programmer-defined symbol may be specified as local (e.g. macros) in the same manner. Because of the local usage of these symbols, they do not appear in the symbol table listing or in the symbol table optionally punched on the object tape.

External, Internal, and Entry Symbols

Programmer-defined symbols may also be used as external, internal, and entry point symbols in addition to their appearance as labels or in assignment statements.

Symbols which fall into one of these three groups are different from other symbols in the program because they can be referenced by other, separately assembled, program modules. The manner in which they are used depends on where they are located: in the program in which they are defined, or in the program in which they are a reference to a symbol defined elsewhere.

If the symbol appears in a program in which it is defined, it must be declared as being available to other programs by the use of the pseudo-ops `.INTERN` or `.ENTRY`.

If the symbol is located in a program in which it is a reference to a symbol defined in another program, it must be declared as external by the use of the `.EXTERN` pseudo-op.

Numbers

Numbers used in a program are interpreted by the assembler according to a radix (number base) specified by the programmer, where the radix may be 2 (binary), 8 (octal), 10 (decimal), or 16 (hexadecimal). The programmer uses the `.RADIX` pseudo-op to set the radix for all numbers which follow. If the `.RADIX` statement is not used, the assembler assumes a radix of 10 (decimal).

The radix may be changed for a single number by appending a radix modifier to the end of the number. These modifiers are B for binary, O or Q for octal, D or . (period) for decimal, and H for hexadecimal. To specify the hexadecimal digits, the letters A through F are used for the

values 10 through 15 decimal. All numbers, however, must begin with a numeral. For example, the following are valid numbers:

10	10 in current radix
10.	10 decimal
10B	10 binary (2 decimal)
0FFH	FF hexadecimal (255 decimal)

The following are invalid numbers:

14B	4 is not a binary digit
FFH	the number must start with a numeral

Arithmetic and Logical Operations

Numbers and defined symbols may be combined using arithmetic and logical operators. The following operators are available:

+	Add (or unary plus)
-	Subtract (or unary minus)
*	Multiply
/	Integer division (remainder discarded)
@	Integer remainder (quotient discarded)
&	Logical AND
!	Logical inclusive OR
^	Logical exclusive OR (or unary radix change)
#	Logical unary NOT
<	Left binary shift
>	Right binary shift

The assembler computes the 16-bit value of a series of numbers and defined symbols connected by these operators. All results are truncated to the left, if necessary. Two's complement arithmetic is used, with the meaning of the sign bit (the most significant bit) being left to the programmer. This means that a numeric value may be either between 0 and 65,535 or between -32,768 and 32,767, depending on whether it is signed or unsigned.

These combinations of number and defined symbols with arithmetic and logical operators are called expressions. When evaluating an expression, the assembler performs the specified operations in a particular order, as follows:

1. Unary minus or plus (- +)
2. Unary radix change (^B ^O ^Q ^D ^H)
3. Left and right binary shift (< >)
4. Logical operators (& ! ^ #)
5. Multiply/Divide (* /)
6. Remainder (@)
7. Add/Subtract (+ -)

Within each of the above groups, the operations are performed from left to right. For example, in the expression:

`-ALPHA+3*BETA/DELTA&^H55`

the unary minus of ALPHA is done first, then DELTA is ANDed with a hexadecimal 55, then BETA is multiplied by 3, the result of which is divided by the result of the AND, and finally, that result is added to the negated ALPHA.

To change the order in which the operations are performed, parentheses may be used to delimit expressions and to specify the desired order of computation. Each expression within parentheses is considered to be a single numeric value, and is completely evaluated before it is used to compute any further values. For example, in the expression:

`4*(ALPHA+BETA)`

the addition of ALPHA to BETA is performed before the multiplication

Radix Change Operator

The radix change operator is used to temporarily change the radix in which a following number or expression is to be interpreted. It is written as an up-arrow (^) followed by the radix modifier of the desired radix. These modifiers are the same as those used to specify the radix of a single number (B-binary, O or Q-octal, D-decimal, and H-hexadecimal). The radix change only affects the immediately following number or parenthesized numeric expression. For example, all of the following are valid representations of the decimal number 33:

`33.`
`33D`
`^D33`
`^D(10*3+3)`
`^D(10*THREE+THREE)`
`^D10*^D3+^D3`

but the following is not a representation of decimal 33 if the prevailing radix is not decimal:

`^D3*10+3`

because the radix change only affects the value immediately following it, in this case 3.

Binary Shifting

The binary shift operators (< left, > right) are used to logically shift a 16-bit value to the left or right. The number of places to be shifted is specified by the value following the shift operator. If that value is negative, the direction of the shift is reversed. For example, all of the following expressions have a value of 4 decimal:

```
8>1
1<2
2>-1
```

One-byte Values

All of the above discussion has been based on the computation of 16-bit (two byte) numeric values. Many of the Z80 operations require an 8-bit (one byte) value. Since all computations are done as a 16-bit value, an operation calling for only eight bits will discard the high order eight bits (the most significant byte) of the value. If the byte discarded is not either zero or minus one (all one bits), a warning will be given on the assembly listing.

Character Values

To generate a binary value equivalent to the ASCII representation of a character string, the single (') or double (") quotation mark is used. The character string is enclosed in a pair of the quotation marks. For example, all of the following are valid character values:

```
"A"
'B'
"AB"
'CD'
```

Note that whichever quotation mark is used to initiate the character string must be used to terminate it. If the string is longer than two bytes, it is truncated to the left. Each 7-bit ASCII character is stored in an 8-bit byte, with the high-order bit set to zero.

A character string of this type may be used wherever a numeric value is allowed.

A single quote may be used inside a string delimited by double quotes, and vice-versa. If it is necessary to use a single quote within a string delimited by single quotes, two single quotes must be used. The same is true for a double quote in a string delimited by double quotes.

Location Counter Reference

The location counter may be referenced as a numeric 16-bit value by the use of the symbol . (period). The value represented by . is always the location counter value at the start of the current assembly language statement. For example:

JMP ..

is an effective error trap, jumping to itself continuously.

Chapter 2

Addressing and Relocation

Address Assignment

As source statements are processed by the assembler, consecutive memory addresses are assigned to the instruction and data bytes of the object program. This is done by incrementing an internal program counter each time a memory byte is assigned. Some statements may increment this internal counter by only one, while others could increase it by a large amount. Certain pseudo-ops and direct assignment statements have no effect on the counter at all.

In the program listing generated by the assembler, the address assigned to every statement is shown.

Relocation

The TDL Z80 Assembler will create a relocatable object program. This program may be loaded into any part of memory as a function of what has been previously loaded. To accomplish this, certain 16-bit values which represent addresses within the program must have a relocation constant added to them. This relocation constant, added when the program is loaded into memory, is the difference between the memory location an instruction (or piece of data) is actually loaded into, and the location it was assembled at. If an instruction had been assembled at location 100 (decimal), and was loaded into location 1100 (decimal), then the relocation constant would be 1000 (decimal).

Not all 16-bit quantities must be modified by the relocation constant. For example, the instruction:

```
LXI H,00FFH
```

references a 16-bit quantity (00FFH) which does not need relocation. However, the set of instructions:

```
JZ DONE
```

```
  .  
  .  
  .
```

```
DONE:
```

does reference a 16-bit quantity (the address of DONE) which must be relocated, since the physical location of DONE changes depending on where the program is loaded into memory.

To accomplish this relocation, the 16-bit value forming

an address reference is marked by the assembler for later modification by the loader. Whether a particular 16-bit value is so marked depends on the evaluation of the arithmetic expression from which it is obtained. A constant value (integer) is absolute (not relocatable), and never modified. Point references (.) are relocatable (assuming relocatable code is being generated), and are always modified by the loader. Symbolic references may be either absolute or relocatable.

If a symbol is defined by a direct assignment statement, it may be absolute or relocatable depending on the expression following the equal sign (=). If the symbol is a label (and relocatable code is being generated) then it is relocatable.

To evaluate the relocatability of an expression, consider what happens at load time. A relocation constant, r , must be added to each relocatable element, and the expression evaluated. For example, in the expression:

$$Z = Y + 2 * X - 3 * W + V$$

where V , W , X , and Y are relocatable. Assume that r is the relocation constant. Adding this constant to each relocatable term, the expression becomes:

$$Z(r) = (Y+r) + 2 * (X+r) - 3 * (W+r) + (V+r)$$

By rearranging the expression, the following is obtained:

$$Z(r) = Y + 2 * X - 3 * W + V + r$$

This expression is suitable for relocation because it contains only a single addition of the relocation constant r . In general, if the expression can be rearranged to result in the addition of either of the following, it is legal:

0*r absolute expression
1*r relocatable expression

If the rearrangement results in the following, it is illegal:

$n * r$ where n is not 0 or 1

Also, if the expression involves r to any power other than 1, it is illegal. This leads to the following rules:

1. Only two values of relocatability for a complete expression are allowed (ie. $n * r$ where $n = 0$ or 1).
2. Division by a relocatable value is illegal.
3. Two relocatable values may not be multiplied together.
4. Relocatable values may not be combined by logical

operators.

5. A relocatable value may not be logically shifted.

If any of these rules is broken, the expression is illegal and an error message is given.

If X, Y, and Z are relocatable symbols, then:

X+Y-Z	is relocatable
X-Z	is absolute
X+7	is relocatable
3*X-Y-Z	is relocatable
4&X-Z	is illegal

Only 16-bit quantities may be relocated. All 8-bit values must be absolute or an error will be given.

Chapter 3

Pseudo-Operations

Pseudo-operations (pseudo-ops) are directions to the assembler to perform certain operations for the programmer, as opposed to machine operations which are instructions to the computer. Pseudo-ops perform such functions as listing control, data conversion, or storage allocation.

Address Mode and Origin

The TDL Z80 Assembler normally assembles programs in relocatable mode, so that the resultant program can be loaded anywhere in memory for execution. Therefore, all programs are assembled assuming their first byte is at address zero (0), because they can be relocated anywhere. When desired, however, the assembler will generate absolute object code, either for the entire program, or just selected portions. The assembler will also locate the assembled code at any address desired. The two pseudo-ops which control both address mode and address origin are .LOC and .RELOC.

.LOC n

This statement sets the location counter to the value n, which may be any valid expression. If n is an absolute value, then the assembler will assign absolute addresses to all of the instructions and data which follow. If n is relocatable, then relocatable addresses will be assigned.

The program is assumed to start with an implicit .LOC to relocatable address zero (0). A program can contain more than one .LOC, each controls the assignment of addresses to the statements following it.

To reset the program counter to its value prior to the last .LOC, the statement:

.RELOC

is used. This statement restores both the value and the addressing mode which were in effect before the immediately preceding .LOC. If no .LOC has been done, then a .RELOC is equivalent to a:

.LOC 0

When in relocatable addressing mode, the assembler will determine whether each 16-bit value is absolute or relocatable as described in Chapter 2.

Data Definition

The TDL 280 Assembler provides a number of different pseudo-ops for describing and entering data to be used by the program.

.RADIX

When the assembler encounters a number in a statement, it converts it to a 16-bit binary value according to the radix indicated by the programmer. The statement:

.RADIX n

where n is 2, 8, 10, or 16, sets the radix to n for all numbers which follow, unless another .RADIX statement is encountered, or the radix is modified by the "r operator or a suffix radix modifier.

The statement:

.RADIX 10

implicitly begins each assembly program, setting the initial radix to decimal.

.BYTE

To enter one (or more) 8-bit (one byte) data values into the program, the statement:

.BYTE n {, n ...}

where n is any expression with a valid 8-bit value is used. More than one byte can be defined at a time by separating it from the preceding value with a comma. All of the bytes defined in a single .BYTE statement are assigned consecutive memory locations. For example:

.BYTE 23,4*~H0FF,BETA-ALPHA

defines three sequential bytes of data.

.WORD

To enter a 16-bit (two byte) value into the program, the statement:

.WORD nn {, nn ...}

where nn is any expression with a valid 16-bit value, is used. Multiple 16-bit values may be defined with one .WORD statement by separating each from the preceding one with a comma.

All 16-bit values defined by the .WORD pseudo-op

are stored in standard Z80 word format, least significant byte first.

For example, the following statement:

```
.WORD ALPHA,234*BETA,"H0EEFF
```

defines three sequential 16-bit values, or a total of six bytes of data.

.ASCII, .ASCIIZ, and .ASCIS

To enter strings of text characters into the program, one of the statements:

```
.ASCII dtextd | [n]  
.ASCIIZ dtextd | [n]  
.ASCIS dtextd | [n]
```

is used. The d represents a text delimiter, and may be any character (other than space or tab) not contained in the text itself. Each character in the text is converted to its 7-bit ASCII representation (with the eighth bit zero), and stored in sequential memory locations. When the delimiter character is again encountered, the text is considered terminated (the delimiter is not stored with the string). The delimited string may be followed by another delimiter, and another string, and this may be repeated as desired.

If it is necessary to include values in the text string for which no character exists, then the second option shown above may be used. If in place of a string delimiter, the assembler finds a left square bracket ([), then the numeric expression enclosed within it and a matching right square bracket (]) is evaluated as an 8-bit value and stored as the next byte of the string. These 8-bit values may be intermixed with delimited strings as required.

It is important to note that tab, carriage return, and line feed are all valid characters within a delimited text string. It is therefore possible that a .ASCIS statement will encompass more than one line in the source program.

The difference between the three pseudo-ops described above is in their treatment of the last byte generated by the statement. The .ASCII statement just stores the byte. The .ASCIIZ statement stores one additional byte after the last one, a null (zero) byte to mark the end of the string in memory. The .ASCIS pseudo-op sets the high-order (eighth) bit of the last byte to one to flag the last byte.

The following are all valid .ASCIS statements:

```
.ASCII /This is a string/  
.ASCII2 /This is two/ ' strings in one place'  
.ASCIS [^HOD] [^HOA] "Message on new line"  
.ASCII \  
Message on new line\  

```

.RAD40

The Radix-40 character set for symbols was chosen because it allows a six character symbol to be stored in only four bytes of memory. To allow the program to define data bytes in this character set, the statement:

```
.RAD40 symbol1 {, symbol2 ...}
```

is used. The symbol must conform to all the rules specified for assembler symbols, and is converted into the Radix-40 notation and stored in four sequential bytes of memory. If multiple symbols are to be converted and stored, each must be separated from the preceding one by a comma.

Storage Allocation

The TDL Z80 Assembler allows the programmer to reserve single locations, or blocks of many locations, for use during the execution of the program. The two pseudo-ops used for this purpose are .BLKB and .BLKW. The format of the statement using these pseudo-ops is:

```
.BLKx n
```

where n is the number of storage locations to be reserved.

For the .BLKB pseudo-op, each storage location consists of one byte, so the above statement will reserve n contiguous bytes of memory, starting at the current location counter. The .BLKW pseudo-op uses a word (two bytes) as its storage unit, so the above statement would reserve n words, or two times n bytes of contiguous memory.

For example, each of the following statements reserves 24 (decimal) bytes of storage:

```
.BLKB 24,  
.BLKW ^D12  
.BLKB 2*12.
```

Program Termination

Every program must be terminated by a .END pseudo-op. The format of this statement is:

```
.END start
```

where start is optional an starting address for the program. The starting address is normally only necessary for the main program. Subprograms, which are called from the main program, need no starting address.

When the assembler encounters the .END pseudo-op during pass 1 of the assembly, it returns to the initialization point to await further instructions (see Appendix C). On a listing pass, the .END pseudo-op initiates the printing of the symbol table (if not suppressed by a prior .XSYM pseudo-op). On a punching pass, the .END pseudo-op punches the EOF record on the object tape.

Subprogram Linkage

Programs usually consist of a main program and numerous subroutines which communicate with each other through parameter linkages and through reference to symbols defined elsewhere in the program. Since the TDL 280 Assembler provides the means for the various program components to be assembled separately from each other, the linking loader (which finally puts the pieces together) must be able to identify those symbols which are references (or referenced) external to the current program. For a given subprogram, these "linkage" symbols are either symbols defined internally which must be available to other programs to reference, or symbols used internally but defined externally to the program. Symbols defined within the program but available to other subprograms are called "internal" symbols. Symbols used internally but defined elsewhere are called "external" symbols.

To set up these linkages between subprograms, three pseudo-ops are provided: .EXTERN, .INTERN, and .ENTRY. All three statements have the same format:

```
.EXTERN symbol1 {, symbol2 ...}  
.INTERN symbol1 {, symbol2 ...}  
.ENTRY symbol1 {, symbol2 ...}
```

where symbol1 is the symbol being declared as external, internal, or as an entry point. Multiple symbols may be declared in the same statement by separating each from the preceding one with a comma.

The .EXTERN statement identifies symbols which are defined elsewhere. External symbols must not be defined within the current subprogram. The external symbols may only be used as addresses, or in expressions that are to be used as addresses. External symbols may be used in the same manner as any other relocatable symbol, with the following restrictions:

1. External symbols may not be used in a .LOC statement.
2. The use of more than one external symbol in a single

expression is illegal. Thus $X+Y$ where X and Y are both external is illegal.

3. Externals may only be additive. Therefore the following expressions are illegal (where X is an external symbol):

-X
 $2 * X$
SQR-X
 $2 * X - X$

The .INTERN pseudo-op identifies those symbols within the current subprogram which are to be made accessible to other programs as external symbols. This statement has no effect on the assembly process for the current program, but merely records the name and value of the identified symbols on the object tape for later use by the linking loader. An internal symbol must be defined within the current program as a label, or in a direct assignment statement.

The .ENTRY pseudo-op functions identically to the .INTERN pseudo-op, with one addition. It is sometimes desirable to put many subroutines with common usage into one "library", and to allow the linking loader to select only those programs from the library which are called by the program being loaded.

The .ENTRY statement, in addition to functioning as a .INTERN statement, also flags the specified symbols as program entry points. If the subprogram is later put into a library, this will specify to the loader that this program is to be loaded only if one of its entry points is referenced as an external symbol by an already loaded program.

Since these entry points are external to the program referencing them, they must be listed in a .EXTERN statement in the calling program.

Listing Control

Program listings are printed on the list device during pass 2 and 4 (see Appendix C) of the assembly. The listing is printed as the source program statements are processed during the pass. The standard listing contains (from left to right):

1. Error flags (if present).
2. Location counter for the first byte generated by this statement.
3. Instruction or data in hexadecimal (maximum of five bytes per line printed).
4. Exact image of the input statement.

The standard listing displays all 16-bit quantities in 16-bit (two byte), most significant byte first, format.

These quantities are properly reversed in the object code as required by the Z80. A 16-bit relocatable address is flagged with an apostrophe ('), and an external reference is flagged with an asterisk (*).

Within a macro expansion, only the macro call and those statements which generate actual object code are normally listed.

If a single statement generates more than the maximum of five bytes that can be listed on a single line, the remaining bytes are properly generated, but not normally listed.

A listing always begins at the top line of the page, and 60 lines are printed per page, with a two line margin at the top, and a two line margin at the bottom. A page is assumed to be 72 (or 80) columns wide (depending on the list device selected - see Appendix C). Each page is numbered, and can have an optional title and sub-title.

The standard listing options can be changed and expanded by the use of the following pseudo-operations:

- .PAGE This statement causes the assembler to skip to the top of the next page (by counting lines). A form feed character in the input text will have the same effect.
- .XLIST This statement causes the assembler to stop listing the assembled program at this point.
- .LIST This statement is normally used following a .XLIST to resume program listing.
- .LALL This statement causes the assembler to list everything which is processed. This includes all text, macro expansions, and all other statements normally suppressed in the standard listing.
- .XALL This statement is normally used following a .LALL to resume the normal listing.
- .SALL This statement causes the suppression of all macro expansions and their text. It can be reset by a subsequent .LALL or .XALL.
- .XSYM This statement suppresses the symbol table listing normally performed upon encountering the .END statement.
- .LSYM Normally not used, this statement enables the listing of the symbol table previously suppressed by the .XSYM pseudo-op.

- .LADDR This statement causes the assembler to list all 16-bit quantities in the same order it generates them in the object code (least significant byte first).
- .XADDR Normally used following a .LADDR statement, this statement resumes the normal listing of 16-bit quantities in non-swapped format.
- .LIMAGE This statement causes the assembler to list every byte generated, even if more than one line (at five bytes per line) is required. In this mode, the assembler will attempt to split the input source statement to indicate which part of the statement is generating which bytes.
- .XIMAGE Normally used following a .LIMAGE statement, this statement resumes the normal listing of only five bytes of generated data per statement.
- .TITLE dtextd This statement defines the delimited string text to be the title to be printed at the top of every page of the listing. The text must be delimited in the same manner as in the .ASCII pseudo-op, and must be no longer than 72 characters. If the .TITLE pseudo op is the first statement on a page, then the new title will be printed at the top of that page.
- .SBTTL dtextd This statement defines the delimited string text to be the sub-title to be printed at the top of every page of the listing. It follows the same rules as the .TITLE pseudo-op.
- .REMARK dtextd This statement inserts a remark into the listing. The delimited text can be any number of lines long, being terminated only by the matching delimiter.
- .PRNTX dtextd This statement, when encountered, causes the delimited text string to be typed on the console. This statement is frequently used to print out conditional information, and to report the progress through pass 1 on very long assemblies.

Punch Control

The TDL Z80 Assembler normally produces an object tape in the TDL Standard Relocatable Format (see Appendix E). However, the assembler can produce an object tape compatible with the "INTEL Standard" hex tape. To control which format is being produced, the two pseudo-ops .PREL and .PABS are used. The .PABS pseudo-op causes the assembler to produce an INTEL compatible tape for all following generated code. The .PREL causes the assembler to return to producing TDL Standard Object Tape.

Every program starts with an implicit .PREL pseudo-op.

Conditional Assembly

Parts of a program may be assembled on a conditional basis depending on the results of certain tests specified to the assembler through the use of the .IFx pseudo-op.

The general form of the pseudo-op is:

.IFx arg,[text]

where the text within the square brackets is assembled only if the specified test on the argument is TRUE.

The first set of conditions which can be tested are the numeric value of the argument. These pseudo-ops are listed below:

.IFE n,[...]	TRUE if n=0 or n=blank
.IFN n,[...]	TRUE if n<0 or n>0
.IFG n,[...]	TRUE if n>0
.IFGE n,[...]	TRUE if n>0 or n=0
.IFL n,[...]	TRUE if n<0
.IFLE n,[...]	TRUE if n<0 or n=0

The following .IF pseudo-ops test for whether the assembler is processing pass 1 or not:

.IF1 ,[...]	TRUE if it is pass 1
.IF2 ,[...]	TRUE if it is not pass 1

The next set of conditionals tests for whether a symbol has been defined yet or not:

.IFDEF symbol,[...]	TRUE if the symbol is defined
.IFNDEF symbol,[...]	TRUE if the symbol is undefined

The next set of .IF pseudo-ops tests to see whether its argument is blank or not. These pseudo-ops require that the argument be enclosed in square brackets ([]). The format is as follows:

.IFB [...],[...] TRUE if blank
.IFNB [...],[...] TRUE if not blank

The quantity enclosed in the brackets is blank if it is empty, or consists only of spaces and tabs. Optionally, the argument being tested may be enclosed in paired delimiters in the same manner as the .ASCII pseudo-ops. If the first non-blank, non-tab, character after the pseudo-op is a left square bracket ([), the bracket method is used, otherwise, the delimiter method. For example:

.IFB /.../, [...]

The last pair of conditionals operate on character strings. They take two arguments which are interpreted as 7-bit ASCII character strings, and make a character by character comparison of the two strings to determine if the condition is met. Each of the strings may either be enclosed in square brackets or delimited by a character, as in the .IFB/.IFNB pseudo-ops above. The same method need not be used for both strings. The format of these conditionals is as follows:

.IFIDN [...] [...],[...] TRUE if identical
.IFDIF [...] [...],[...] TRUE if different

The maximum length of the strings to be compared is 255 characters.

Synonyms

It sometimes becomes useful, for documentation or ease of programming, to define new names for already existing symbols. The TDL Z80 Assembler has four pseudo-ops which allow the definition of synonyms for already defined symbols. The format of these pseudo-ops is:

.xxSYN symbol1,symbol2

The four pseudo-ops are .SYN, .OPSYN, .SYSYN, and .MASYN. The only difference between the four is that the latter three limit the type of symbol for which the synonym is being defined.

The statement above defines the second operand as being synonymous with the first operand. In the case of the .SYN pseudo-op, the symbol tables are searched for the first operand in the order: programmer defined symbol, macro, operation. The .OPSYN pseudo-op limits the search to operations, the .SYSYN to programmer defined symbols, and the .MASYN to macros. The second operand is defined to be identical to the first operand at the time the synonym is defined. Later changes to the first operand will not affect

the second.

The following are valid synonym definitions:

```
.OPSYN .BYTE,DB  
.SYN .WORD,DW  
.SYSYN ALPHA,BETA  
.SYN A,R1
```

Object Machine Validation

Although the TDL Macro Assembler will run only on a Z80 processor, it can obviously be used to generate object code for any of the 8080 compatible micro-processors. To facilitate the use of the assembler for this purpose, two additional pseudo-ops are available: .I8080 and .Z80.

The .I8080 pseudo-op causes all subsequent uses of machine operations which are unique to the Z80 (and hence unavailable on the 8080) to be flagged with a Z warning message. Such uses will be properly assembled however.

The .Z80 pseudo-op (which is the default) disables the feature so that no further Z warnings will be given.

Chapter 4

Macros

A common characteristic of assembly language programs is that many coding sequences are repeated over and over with only a change in one or two of the operands. It is convenient, therefore, to provide a mechanism by which the repeated sequences can be generated by a single statement. The TDL Z80 Assembler provides the capability to do so by allowing the repeated sequences to be written, with dummy values for the changed operands, as a macro. A single statement, referring to the macro by name and providing values for the dummy operands, can then generate the repeated sequence.

Macro Definition

A macro is defined by use of the .DEFINE pseudo-op. This is followed by the symbolic name of the macro. The macro name must follow the rules for the construction of symbols. The name may be followed by a list of dummy arguments enclosed in square brackets. The dummy arguments are separated by commas, and may be any symbol which is convenient. Following the macro name and optional dummy arguments must be an equal sign (=). The following are examples of the heading part of a macro definition:

```
.DEFINE MACRO =  
.DEFINE MOVE[A,B] =  
.DEFINE BIGMAC[ARG1,ARG2,ARG3,%ARG5] =
```

Following the macro definition header comes the body of the macro. It need not start on the same line as the definition header. The body of the macro is delimited by a matched pair of left and right square brackets ([]). For example:

```
.DEFINE MOVE[A,B]=  
[LDA A  
STA B]
```

Macro Calls

A macro may be called by any statement. A macro call consists of the macro name followed (optionally) by a list of arguments. The arguments are separated by commas, and may optionally be enclosed in left and right square brackets ([]). If the brackets are used (the first non-blank/non-tab character after the macro name is a left square bracket),

then the arguments are terminated by a right square bracket. If there are n dummy arguments in the macro definition, then all arguments after the first n are ignored (although they do take space and time to process). If the brackets are omitted, the argument string ends when a carriage return or semicolon is encountered.

The arguments must be written in the order in which they are to be substituted for the dummy arguments. The first argument is substituted for each appearance of the first dummy argument, the second for the second, etc. The actual arguments are substituted as character strings for the dummy arguments, no evaluation of the arguments takes place until the macro is processed.

Referring to the definition of MOVE above, the occurrence of the statement:

```
MOVE ALPHA,BETA
```

will cause the substitution of ALPHA for A and BETA for B in the macro.

Statements which contain macro calls may be labelled and have comments like any other statement.

Macro arguments are terminated only by comma, carriage return, semicolon, or right square bracket (when started by left square bracket). These characters may not be used in the arguments unless the argument is enclosed in parentheses. Each time an argument is passed to a macro, one set of matched parentheses is removed, but all of the characters within the parentheses are substituted for the dummy argument in the macro. Note that spaces and tabs do not terminate arguments, but are considered to be part of them.

Macros do not need to have arguments. The macro name (and arguments if any) may appear anywhere in a statement where a symbol would normally appear, and the text of the macro exactly replaces the macro name and its arguments in that statement.

Comments

Comments may be included within a macro definition. Storing the comments with the macro (so that they will appear when the macro is expanded) takes space however. If the comment within the macro definition is preceded by two semicolons (instead of the normal one), the comment will be ignored during the definition of the macro, and will not be stored as part of the definition. This will eliminate the appearance of the comment every time the macro expansion is listed, however.

Created Symbols

When a macro is called, it is often useful to generate symbols without explicitly stating them in the call. A good example of this is labels within the macro body. It is usually not necessary to refer to these labels externally to the macro expansion, therefore there is no reason why the programmer should be concerned as to what those labels are. The same with temporary data areas. To avoid conflicts, however, it is necessary that a different symbol be used each time the macro is called (even with local symbols, the macro could be called more than once between two global symbols). Created symbols are used for this purpose.

Each time a macro that requires a created symbol is called, a symbol is generated and inserted into the macro. These symbols are of the form ..nnnn (two periods followed by four digits). It should be noted that this makes these symbols local symbols (start with two periods). The programmer is advised not to use symbols of this form. The four digits start at 0000 and are incremented by one each time a symbol is created.

A created symbol is specified in the macro definition by preceding a dummy argument by a percent sign (%). When the macro is called, all dummy arguments of the form %symbol are replaced by created symbols (each with a different one). If, however, the position of the dummy argument in the argument list corresponds to an actual argument provided in the call, then the actual argument is used in place of the created one.

An actual argument can in fact be empty (signified by two consecutive commas in the argument list). An argument of this kind (a "null" argument) is considered to be defined as having a value of the empty string (no characters), and will prevent the generation of a created symbol for its corresponding dummy argument.

For example:

```
.DEFINE PRINT[A,%B]=  
[CALL LINPRT  
JMP %B  
.ASCIS \A\  
%B:]
```

This macro prints a message on the printer. The first argument to the macro is the text string to be printed. LINPRT is a line printer routine. Labelling the location following the text is necessary because of the indeterminate length of the message. The use of a created symbol here is useful since there would normally be no reason to reference the label. Calling the macro by:

PRINT This is the message

would result in printing "This is the message" when the assembled macro was executed. If it had been called:

PRINT This is the message,MAIN

the message would have been printed, but control would be transferred to the label MAIN, which substituted for %B instead of a created symbol.

Concatenation

The apostrophe or single quote (') is defined within a macro definition as the concatenation operator. This allows a macro argument to be only part of a symbol or expression, with the character string which is substituted for the dummy argument being joined with other character strings that are part of the macro definition to form a complete symbol or expression. This joining is called concatenation. Concatenation is performed by the assembler when an apostrophe is used between the strings to be joined (one or both of which must be a dummy macro argument). For example:

```
.DEFINE BR[A,B]=  
[JR'A B]
```

defines a conditional branch statement. When called, the argument A is appended to the JR to form a single symbol. If the call were:

BR Z,LOOP

then the generated code would be:

JRZ LOOP

Default Arguments

Normally, missing arguments in a macro are replaced by nulls. For example, in the macro:

```
.DEFINE BYTES[A1,A2,A3,A4,A5,A6]=  
[.BYTE A1,A2,A3,A4,A5,A6]
```

a call of BYTES[1,2] would generate an error because of the missing arguments to the pseudo-op .BYTE.

To remedy this, the assembler provides the programmer with the means to supply default arguments to be used when no argument is provided in the macro call. Default argument are defined as part of the macro definition by enclosing them in parentheses and inserting them immediately after the

dummy argument to which they refer. To solve the above problem, the definition would be written as:

```
.DEFINE BYTES[A1(0),A2(0),A3(0),A4(0),A5(0),A6(0)]=  
[.BYTE A1,A2,A3,A4,A5,A5]
```

which would always generate six bytes of data, regardless of how many arguments were provided in the call.

Nesting

Macros may be nested. This means that macros may be both called and defined within other macros. A macro that is defined within another macro may not be called until the defining macro has been called. At that time, the new macro is available to be called by any statement.

The only limit to how many levels deep macro calls and definitions may be nested is the amount of memory available.

Appendix A

Summary of Machine Operation Mnemonics

The following section presents a summary of the Z80 machine operations and their assembler mnemonics. The appendix is arranged by type of instruction for ease of reference. For further information on the machine operations, refer to the "ZILOG Z80-CPU Technical Manual".

To make the information presented more readily usable, a shorthand notation is used for describing the assembler format of the instruction and its actual operation. All capital letters and special characters in the mnemonic description are required. The lower case letters indicate a class of values which can be inserted in the instruction at that point. A single lower case letter indicates an 8-bit quantity or register, while a double lower case letter indicates a 16-bit quantity or register. A symbol enclosed in parentheses in the machine operation section indicates that the value whose address is specified is used. The following is a summary of the notation used; exceptions will be noted where appropriate in the following sections.

r one of the 8-bit registers A, B, C, D, E, H, L
n any 8-bit absolute value
ii an index register reference, either X or Y
d an 8-bit index displacement where $-128 < d < 127$
zz B for the BC register pair, D for the DE pair
nn any 16-bit value, absolute or relocatable
rr B for the BC register pair, D for the DE pair, H for the HL pair, SP for the stack pointer
qq B for the BC register pair, D for the DE pair, H for the HL pair, PSW for the A/Flag pair
s any of r (defined above), M, or d(ii)
IFF interrupt flip-flop
CY carry flip-flop
ZF zero flag
tt B for the BC register pair, D for the DE pair, SP for the stack pointer, X for index register IX
uu B for the BC register pair, D for the DE pair, SP for the stack pointer, Y for index register IY
b a bit position in an 8-bit byte, where the bits are numbered from right to left 0 to 7
PC program counter
v[n] bit n of the 8-bit value or register v
v[n-m] bits n through m of the 8-bit value or register v
vv\H the most significant byte of the 16-bit value or register vv
vv\L the least significant byte of the 16-bit value or register vv

Appendix A: Summary of Machine Operation Mnemonics

Iv an input operation on port v

Ov an output operation on port v

w<-v the value of w is replaced by the value of v

w<->v the value of w is exchanged with the value of v

8-Bit Load Group

Mnemonic	Operation	# of Bytes
MOV r,r'	r ← r'	1
MOV r,M	r ← (HL)	1
MOV r,d(ii)	r ← (ii+d)	3
MOV M,r	(HL) ← r	1
MOV d(ii),r	(ii+d) ← r	3
MVI r,n	r ← n	2
MVI M,n	(HL) ← n	2
MVI d(ii),n	(ii+d) ← n	4
LDA nn	A ← (nn)	3
STA nn	(nn) ← A	3
LDAX zz	A ← (zz)	1
STAX zz	(zz) ← A	1
LDAI	A ← I	2
LDAR	A ← R	2
STAI	I ← A	2
STAR	R ← A	2

16-Bit Load Group

Mnemonic		Operation	# of Bytes
LXI	rr,nn	rr <- nn	3
LXI	ii,nn	ii <- nn	4
LBCD	nn	B <- (nn+1)	4
		C <- (nn)	
LDED	nn	D <- (nn+1)	4
		E <- (nn)	
LHLD	nn	H <- (nn+1)	3
		L <- (nn)	
LIXD	nn	IX\H <- (nn+1)	4
		IX\L <- (nn)	
LIYD	nn	IY\H <- (nn+1)	4
		IY\L <- (nn)	
LSPD	nn	SP\H <- (nn+1)	4
		SP\L <- (nn)	
SBCD	nn	(nn+1) <- B	4
		(nn) <- C	
SDED	nn	(nn+1) <- D	4
		(nn) <- E	
SHLD	nn	(nn+1) <- H	3
		(nn) <- L	
SIXD	nn	(nn+1) <- IX\H	4
		(nn) <- IX\L	
SIYD	nn	(nn+1) <- IY\H	4
		(nn) <- IY\L	
SSPD	nn	(nn+1) <- SP\H	4
		(nn) <- SP\L	
SPHL		SP <- HL	1
SPIX		SP <- IX	2
SPIY		SP <- IY	2
PUSH	qq	(SP-1) <- qq\H	1
		(SP-2) <- qq\L	
		SP <- SP - 2	
PUSH	ii	(SP-1) <- ii\H	2
		(SP-2) <- ii\L	
		SP <- SP - 2	
POP	qq	qq\H <- (SP+1)	1
		qq\L <- (SP)	
		SP <- SP + 2	
POP	ii	ii\H <- (SP+1)	2
		ii\L <- (SP)	
		SP <- SP + 2	

Exchange and Block Transfer and Search Group

Mnemonic	Operation	# of bytes
XCHG	HL <-> DE	1
EXAF	PSW <-> PSW'	1
EXX	BCDEHL <-> BCDEHL'	1
XTHL	H <-> (SP+1) L <-> (SP)	1
XTIX	IX\H <-> (SP+1) IX\L <-> (SP)	2
XTIY	IY\H <-> (SP+1) IY\L <-> (SP)	2
LDI	(DE) <- (HL) DE <- DE + 1 HL <- HL + 1 BC <- BC - 1	2
LDIR	repeat LDI until BC=0	2
LDD	(DE) <- (HL) DE <- DE - 1 HL <- HL - 1 BC <- BC - 1	2
LDDR	repeat LDD until BC=0	2
CCI	A - (HL) HL <- HL + 1 BC <- BC - 1	2
CCIR	repeat CCI until A=(HL) or BC=0	2
CCD	A - (HL) HL <- HL - 1 BC <- BC - 1	2
CCDR	repeat CCD until A=(HL) or BC=0	2

8-Bit Arithmetic and Logical Group

Mnemonic		Operation	# of Bytes
ADD	r	$A \leftarrow A + r$	1
ADD	M	$A \leftarrow A + (HL)$	1
ADD	d(ii)	$A \leftarrow A + (ii+d)$	3
ADI	n	$A \leftarrow A + n$	2
ADC	s	$A \leftarrow A + s + CY$	
ACI	n		
SUB	s	$A \leftarrow A - s$	
SUI	n		
SBB	s	$A \leftarrow A - s - CY$	
SBI	n		
ANA	s	$A \leftarrow A \& s$	
ANI	n		
ORA	s	$A \leftarrow A s$	
ORI	n		
XRA	s	$A \leftarrow A \wedge s$	
XRI	n		
CMP	s	$A - s$	
CPI	n		
INR	s	$s \leftarrow s + 1$	
DCR	s	$s \leftarrow s - 1$	

General Purpose Arithmetic and Control Group

<u>Mnemonic</u>	<u>Operation</u>	<u># of Bytes</u>
DAA	convert A to packed BCD after an add or subtract of packed BCD operands	1
CMA	A ← #A	1
NEG	A ← -A	2
CMC	CY ← #CY	1
STC	CY ← 1	1
NOP	no operation	1
HLT	halt	1
DI	IFF ← 0	1
EI	IFF ← 1	1
IM0	interrupt mode 0	2
IM1	interrupt mode 1	2
IM2	interrupt mode 2	2

16-Bit Arithmetic Group

<u>Mnemonic</u>		<u>Operation</u>	<u># of Bytes</u>
DAD	rr	HL <- HL + rr	1
DADC	rr	HL <- HL + rr + CY	2
DSBC	rr	HL <- HL - rr - CY	2
DADX	tt	IX <- IX + tt	2
DADY	uu	IY <- IY + uu	2
INX	rr	rr <- rr + 1	1
INX	ii	ii <- ii + 1	2
DCX	rr	rr <- rr - 1	1
DCX	ii	ii <- ii - 1	2

Rotate and Shift Group

Mnemonic		Operation	# of Bytes
RLC		A[n+1] <- A[n] A[0] <- A[7] CY <- A[7]	1
RAL		A[n+1] <- A[n] A[0] <- CY CY <- A[7]	1
RRC		A[n] <- A[n+1] A[7] <- A[0] CY <- A[0]	1
RAR		A[n] <- A[n+1] A[7] <- CY CY <- A[0]	1
RLCR	s	s[n+1] <- s[n] s[0] <- s[7] CY <- s[7]	2 (or 4)
RALR	s	s[n+1] <- s[n] s[0] <- CY CY <- s[7]	
RRCR	s	s[n] <- s[n+1] s[7] <- s[0] CY <- s[0]	
RARR	s	s[n] <- s[n+1] s[7] <- CY CY <- s[0]	
SLAR	s	s[n+1] <- s[n] s[0] <- 0 CY <- s[7]	
SRAR	s	s[n] <- s[n+1] s[7] <- s[7] CY <- s[0]	
SRLR	s	s[n] <- s[n+1] s[7] <- 0 CY <- s[0]	
RLD		A[0-3] <- (HL)[4-7] (HL)[4-7] <- (HL)[0-3] (HL)[0-3] <- A[0-3]	2
RRD		(HL)[0-3] <- (HL)[4-7] (HL)[4-7] <- A[0-3] A[0-3] <- (HL)[0-3]	2

Bit Set, Reset, and Test Group

<u>Mnemonic</u>		<u>Operation</u>	<u># of Bytes</u>
BIT	b,r	ZF $\leftarrow$ #r[b]	2
BIT	b,M	ZF $\leftarrow$ #(HL)[b]	2
BIT	b,d(ii)	ZF $\leftarrow$ #(ii+d)[b]	4
SET	b,s	s[b] $\leftarrow$ 1	
RES	b,s	s[b] $\leftarrow$ 0	

Jump Group

Mnemonic		Operation	# of Bytes
JMP	nn	PC <- nn	3
JZ	nn	if zero, then JMP else continue	3
JNZ	nn	if not zero	3
JC	nn	if carry	3
JNC	nn	if not carry	3
JPO	nn	if parity odd	3
JPE	nn	if parity even	3
JP	nn	if sign positive	3
JM	nn	if sign negative	3
JO	nn	if overflow	3
JNO	nn	if not overflow	3
JMPR	nn	PC <- nn where $-126 < nn - PC < 129$	2
JRZ	nn	if zero, then JMPR else continue	2
JRNZ	nn	if not zero	2
JRC	nn	if carry	2
JRNC	nn	if not carry	2
DJNZ	nn	B <- B - 1 if B=0 then continue else JMPR	2
PCHL		PC <- HL	1
PCIX		PC <- IX	2
PCIY		PC <- IY	2

Call and Return Group

<u>Mnemonic</u>		<u>Operation</u>	<u># of Bytes</u>
CALL	nn	(SP-1) <- PC\H (SP-2) <- PC\L SP <- SP - 2 PC <- nn	3
CZ	nn	if zero, then CALL else continue	3
CNZ	nn	if not zero	3
CC	nn	if carry	3
CNC	nn	if not carry	3
CPO	nn	if parity odd	3
CPE	nn	if parity even	3
CP	nn	if sign positive	3
CM	nn	if sign negative	3
CO	nn	if overflow	3
CNO	nn	if not overflow	3
RET		PC\H <- (SP+1) PC\L <- (SP) SP <- SP + 2	1
RZ		if zero, then RET else continue	1
RNZ		if not zero	1
RC		if carry	1
RNC		if not carry	1
RPO		if parity odd	1
RPE		if parity even	1
RP		if sign positive	1
RM		if sign negative	1
RO		if overflow	1
RNO		if no overflow	1
RETI		return from interrupt	2
RETN		return from non-maskable interrupt	2
RST	n	(SP-1) <- PC\H (SP-2) <- PC\L PC <- 8 * n where 0 <= n < 8	1

Input and Output Group

Mnemonic -----		Operation -----	# of Bytes -----
IN	n	A <- In	2
INP	r	r <- I(C)	2
INI		(HL) <- I(C) B <- B - 1 HL <- HL + 1	2
INIR		repeat INI until B=0	2
IND		(HL) <- I(C) B <- B - 1 HL <- HL - 1	2
INDR		repeat IND until B=0	2
OUT	n	On <- A	2
OUTP	r	O(C) <- r	2
OUTI		O(C) <- (HL) B <- B - 1 HL <- HL + 1	2
OUTIR		repeat OUTI until B=0	2
OUTD		O(C) <- (HL) B <- B - 1 HL <- HL - 1	2
OUTDR		repeat OUTD until B=0	2

Appendix B

Summary of Pseudo-Operation Mnemonics

`.ASCII dtextd | [n] ...`

The `.ASCII` pseudo-op enters 7-bit ASCII characters into the program. The text is either entered between two delimiters, or as a numeric value enclosed in square brackets (`[]`), and the two forms may be intermixed and repeated as desired.

`.ASCIS dtextd | [n] ...`

The `.ASCIS` pseudo-op enters 7-bit ASCII characters into the program, and flags the last character by setting its high-order bit on. The format of the text is the same as for the `.ASCII` pseudo-op.

`.ASCIZ dtextd | [n] ...`

The `.ASCIZ` pseudo-op enters 7-bit ASCII characters into the program, and flags the end of the characters by inserting an additional null byte. The format of the text is the same as for the `.ASCII` pseudo-op.

`.BLKB nn`

The `.BLKB` pseudo-op reserves a block of contiguous storage `nn` bytes long.

`.BLKW nn`

The `.BLKW` pseudo-op reserves a block of contiguous storage `nn` words long (`nn x 2` bytes).

`.BYTE n {, n ...}`

The `.BYTE` pseudo-op enters single byte values into the program. Multiple values may be entered by separating them with a comma.

`.DEFINE symbol[arg1,arg2,...]=[text]`

The `.DEFINE` pseudo-op defines a macro with the name `symbol`. `arg1` through `argn` are optional dummy arguments. The body of the macro is represented by `text`.

.END nn

The .END pseudo-op signals the end of the assembly. When encountered during PASS 1, it simply returns to the initialization section. During a listing pass, it initiates the listing of the symbol table (if not previously suppressed by the .XSYM pseudo-op). During a punch pass, it generates an EOF record on the hex tape containing the value nn as the starting address of the object program.

.ENTRY symbol1 {, symbol2 ...}

The .ENTRY pseudo-op identifies the internally defined symbols which are subroutine library entry points to this program. Multiple symbols may be identified by separating them with commas.

.EXTERN symbol1 {, symbol2 ...}

The .EXTERN pseudo-op defines those symbols which are referenced in this program but are defined in another, separately assembled, program. Multiple symbols can be defined by separating them with commas.

.I8080

The .I8080 pseudo-op enables the Z warning message. This warning will be given whenever a machine operation unique to the Z80 is encountered.

.INTERN symbol1 {, symbol2 ...}

The .INTERN pseudo-op identifies those symbols which are defined in this program and which will be referenced as external symbols by some separately assembled program. Multiple symbols may be identified by separating them with commas.

.LADDR

The .LADDR pseudo-op change the listing mode from displaying 16-bit quantities to displaying the Z80 image with the least significant byte first.

.LALL

The .LALL pseudo-op causes the assembler to list every text character processed, including those suppressed in the normal listing.

`.LIST`

The `.LIST` pseudo-op resumes a listing which has been stopped by the `.XLIST` pseudo-op.

`.LIMAGE`

The `.LIMAGE` pseudo-op changes the listing mode to display every byte of object code generated rather than the normal mode of a maximum of five bytes per statement.

`.LOC nn`

The `.LOC` pseudo-op changes the value of the assembler's program counter to `nn`. If `nn` is relocatable, then all labels will be assigned relocatable values. If it is absolute, then absolute values will be assigned.

`.LSYM`

The `.LSYM` pseudo-op reenables the listing of the symbol table during the `.END` pseudo-op processing after it has been disabled by the `.XSYM` pseudo-op. The `.LSYM` pseudo-op must occur prior to the `.END` pseudo-op to be effective.

`.MASYN symbol1,symbol2`

The `.MASYN` pseudo-op allows the definition of a new macro to be the same as a previously defined one. `Symbol2` is defined to be a macro identical to the one defined as `symbol1`.

`.OPSYN symbol1,symbol2`

The `.OPSYN` pseudo-op allows the definition of a new op code mnemonic as a synonym of an already existing one. The `symbol1` must be a defined machine or pseudo op code (or one previously defined using `.OPSYN`), `symbol2` will be defined to be the same operation.

`.PABS`

The `.PABS` pseudo-op signals that the hex object tape produced from this point in the assembly on is to be in absolute (INTEL compatible) format.

.PAGE

The .PAGE pseudo-op causes a skip to the top of the next page during a listing pass.

.PREL

The .PREL pseudo-op signals that the hex object tape produced from this point in the assembly on is to be in relocatable (TDL standard) format.

.PRNTX dtextd

The .PRNTX pseudo-op will cause its text string to be printed on the console whenever it is encountered in the assembly process.

.RADIX n

The .RADIX pseudo-op changes the default base in which a numeric constant is interpreted during the assembly to n. The valid values for n are 2, 8, 10, or 16. The value is always interpreted as a decimal number.

.RAD40 symbol

The .RAD40 pseudo-op generates a unique 4 byte value in radix-40 notation for the symbol given. The symbol must conform to the rules for any symbol in the assembly. This pseudo-op is used mostly for developing system software utilizing symbol tables.

.RELOC

The .RELOC pseudo-op restores the value of the assembler's program counter to whatever it was before the immediately preceding .LOC pseudo-op.

.REMARK dtextd

The .REMARK pseudo-op allow the entry of multiple line comments into the source program. All of the text between the delimiters is listed but is ignored. The text may contain carriage return/line feeds.

.SALL

The .SALL pseudo-op suppresses all macro expansions on the assembly listing (normally all lines generating code are listed).

.SBTTL dtextd

The **.SBTTL** pseudo-op sets the sub-title for the assembly listing to the specified text string (which must be less than 72 characters in length). If the **.SBTTL** pseudo-op is the first operation after a **.PAGE**, the sub-title will appear on the new page.

.SYN symbol1,symbol2

The **.SYN** pseudo-op makes any two symbols synonymous. The symbol tables are searched for symbol1 in the normal operand field order (label/symbol, macro, opcode), and symbol2 is defined to have the same value as symbol1.

.SYSYN symbol1,symbol2

The **.SYSYN** pseudo-op makes one symbol the synonym of an already defined symbol/label. The value of a symbol/label symbol1 is obtained, and symbol2 is defined to be the same type and value.

.TITLE dtextd

The **.TITLE** pseudo-op sets the title for the assembly listing to the specified text string (which must be less than 72 characters in length). The title is put at the top of every page during a listing. If the **.TITLE** pseudo-op is the first operation after a **.PAGE** pseudo-op, the title will be listed on the new page.

.WORD nn {, nn ...}

The **.WORD** pseudo-op enters 2-byte values into the program in proper Z80 format (least significant byte first). Multiple values may be entered by separating them with a comma.

.XADDR

The **.XADDR** pseudo-op is used after a **.LADDR** pseudo-op to return to the standard format of listing 16-bit values.

.XALL

The **.XALL** pseudo-op is used after a **.LALL** or **.SALL** pseudo-op to return to the standard listing mode.

.XIMAGE

The .XIMAGE pseudo-op is used after a .LIMAGE pseudo-op to return to the standard listing mode of only five object bytes per statement.

.XLIST

The .XLIST pseudo-op suppresses the listing of all following statements (until a .LIST pseudo-op is encountered).

.XSYM

The .XSYM pseudo-op disables the listing of the symbol table by the .END pseudo-op (unless reenabled by the .LSYM pseudo-op). The .XSYM pseudo-op must appear before the .END pseudo-op to be effective.

.Z80

The .Z80 pseudo-op is used to disable the effect of a previous .I8080 pseudo-op. This inhibits the Z warning message on machine operations unique to the Z80.

.IFx arg,[text]

The .IFx pseudo-op will assemble the text specified only if the particular condition being tested for is true. The .IFx pseudo-ops and their conditions are as follows:

- .IF1: assembling pass 1
- .IF2: not assembling pass 1
- .IFB: blank
- .IFDEF: defined
- .IFDIF: different
- .IFE: zero or blank
- .IFG: positive
- .IFGE: zero or positive
- .IFIDN: identical
- .IFL: negative
- .IFLE: zero or negative
- .IFN: not zero
- .IFNB: not blank
- .IFNDEF: not defined

Appendix C

Operation of the Assembler

The TDL Z80 Relocating Assembler is designed to operate with the TDL System Monitor. It relies upon the Monitor for all I/O and memory management functions. (For further information on the TDL Monitors, consult the appropriate monitor reference manual.) When operating, the assembler will use all available memory for its various tables (all memory between the end of the assembler and the highest available memory location). No memory location below the assembler is changed by its operation.

The first step in using the assembler is to load it into the desired memory location using the monitor "R" command. After the load has been completed, if the monitor is not located at the standard memory address (F000 hex), it will be necessary to change the assembler's monitor transfer vector to point to the monitor. This transfer vector consists of nine (9) JMP instructions located beginning at relative address six (6 hex) in the program. The addresses of these instructions should be modified to point to the correct locations.

After the assembler is loaded and ready to operate, the appropriate monitor commands should be used to designate the reader, punch, and list devices as desired. The console device is also used during the assembly. After readying the source program in the reader, a "G" command should be used to start the assembler.

It is important to note that the assembler requires a "controlled" reader device (a device which provides characters on demand, at whatever rate the program wants them). In the same manner in which the assembler "waits" for the next character from the reader, the reader must be capable of "waiting" for the next demand from the assembler. (For further information on converting a non-controlled reader to a controlled one, see one of the TDL System Monitor reference manuals.)

When first started (and whenever an assembly pass is completed), the assembler asks "PASS=" on the console. Valid responses to this are only the numbers from 0 to 8. A response of 0 will return to the monitor, but in a manner which will allow resumption of the assembly by reentering the "G" command. The values 1 through 4 signify which assembler pass is desired, as follows:

- 1 signifies the first assembly pass. The source is read, and all necessary tables are built.

- 2 signifies the listing only pass. The source is re-read, and a listing of the assembled program is produced on the list device.
- 3 signifies the punch only pass. The source is re-read, and an object tape of the assembled program is produced on the punch device.
- 4 signifies the combination of passes 2 and 3.

The values of 5 through 8 provide the same options as 1 through 4, but do not reinitialize the assembler in any way before proceeding. This allows the assembly of a program residing on more than one source tape. Each of the pieces must, however, be terminated by its own .END pseudo-op.

During the first assembly pass (pass 1), it is possible that some error messages will be output on the list device. These errors will be those uniquely determined during the pass.

During the punch only pass (pass 3), no error messages will be listed, but an errors indication will be given on the console at the end of the assembly.

While an assembly is taking place, a number of console control options are available. A control-C will always trap back to the monitor after the completion of the current statement. The assembly may be resumed (if no registers have been changed) by using the monitor "G" command. A control-C will, however, result in monitor output on the console device, which could spoil a listing if the console is the list device. To avoid this, the use of a control-S will temporarily halt the assembly (eg. to put more paper in the teletype), but will not return to the monitor or cause any spurious output on the console device. A control-Q will resume the assembly. If a control-C is entered after the control-S, a trap to the monitor will occur as above. All of the above features will, however, be disabled if the reader device is specified as the Teletype.

When starting a listing pass, the paper in the list device should be positioned at the top line of a page. The assembler will count lines and put a page number and heading at the top of every page. The page width is determined by the assigned list device. If the list device is the teletype (AL=T), then the page is assumed to be 72 characters wide. If not, then it is assumed to be 80 characters wide. In either case, it is assumed to be 66 lines long, and a two line margin is left at the top and the bottom of the page.

Appendix D

Error Codes

Errors in the source program encountered during the assembly process are indicated on the listing by a single letter code at the left of the statement in error. Although the assembler may detect more than two errors per statement, only the first two codes are given. As an added aid to locating the error in the statement, a question mark is printed to the right of the character which triggered the error. All errors generate a question mark, even if they are not one of the first two per statement.

The following is a list of the error codes and their meanings:

- A Argument error. This is a broad class of errors which may be caused by many different things.
- B Bad macro error. Either an error in a macro definition or a call on a bad macro.
- D Duplicate symbol reference error. The symbol flagged is multiply-defined. The first value given to the symbol is used in the assembly.
- E External symbol error. An external symbol is improperly used in the statement.
- I Implementation error. The pseudo-operation used in the statement has not been implemented yet.
- M Multiply-defined symbol error. A symbol is defined more than once. This error is given mostly during Pass 1. During the other passes, it usually will appear as a phase error (P).
- O Operation error. The symbol in the operation field is not a valid machine operation code, macro name, or symbol.
- P Phase error. A label is assigned a value during Pass 2 (or 3 or 4) which is different than that assigned during Pass 1.
- Q Questionable error. This is a broad class of warnings which the assembler gives when it finds ambiguous statements. Q errors may or may not generate correct code. The assembler will attempt to do what the programmer intended.

- R Relocation error. A relocatable symbol or expression is incorrectly used (eg. in a .BLKB pseudo-op).
- T Table overflow. One of the Assembler's internal tables has overflowed. The Assembler will attempt to continue, but no new labels or macros will be defined.
- U Undefined label/symbol error. A symbolic reference which was never defined is used in the statement.
- X Index error. Another character appears in a statement at a point where only an index register reference is allowed (X or Y).
- Z Z80 error. A Z80 machine operation has been encountered while in 8080 mode (.I8080). This is only a warning and the opcode will be properly assembled.

Appendix E

Object Tape Formats

The TDL Assembler produces two different object tape formats depending on the use of the .PABS and .PREL pseudo-ops.

The use of the .PREL pseudo-op (which is the default if neither is specified) causes the generation of the TDL Standard Relocatable Object Tape. This format allows the relocating loaders provided as part of the TDL System Monitors to load the program for execution anywhere in memory.

The object tape consists of one or more blocks of 7-bit ASCII characters in the following format:

Character	Contents	Meaning
-----	-----	-----
0-1	CR-LF	carriage return/line feed to separate the blocks and to provide a more readable listing of the object tape if desired.
2	;	semi-colon to act as flag indicating the start of a relocatable block.
3-4	hex digits	in this field (and all those that follow) two ASCII characters are used to represent one byte of binary information. These characters are 0-9 and A-F (the same as those used by the assembler for hexadecimal numbers). The byte whose value is given by these two characters contains the number of bytes of program (or data) which are contained in this block of the tape.
5-8	hex digits	these two bytes contain the load address for the rest of the program in the block (the address in memory at which the information is to be put). The address is punched most significant byte first for readability.
9-10	hex digits	this byte contains the relocation flag for the load address. If it is 0 then the load address is used as

given (absolute). If it is 1 then the load address is modified by the relocation factor specified to the loader (relocatable).

- | | | |
|---------|------------|--|
| 11-12 | hex digits | this byte contains the relocation control bits for the next eight program/data bytes. Each bit in the byte control the relocation of one of the following bytes, proceeding from left to right (bit 7 controls the first byte, etc.). A 0 bit means that the byte is to be loaded unchanged (absolute). A 1 bit followed by a 0 bit means that the following two bytes are a relocatable address, and the specified relocation factor is to be added to them. A 1 bit followed by a 1 bit specifies a linking loader control function (For further information refer to the TDL Linking Loader User's Manual). |
| 13-28 | hex digits | these bytes contain the data to be loaded under the control of the preceding relocation control bits. The total number of program/data bytes is obtained from the initial count field at the start of the block. After eight of these bytes, the two fields (relocation control and data) are repeated until the count is exhausted. |
| n-(n+1) | hex digits | this byte contains the block checksum. This is the two's complement of the sum of all of the previous bytes in the block (starting with the count byte). This guarantees that the sum of all the bytes in a properly read block will be zero. |

The use of the .PABS pseudo-op causes an INTEL "hex" object tape to be produced. This tape can also be loaded by the TDL System Monitors. This object tape has a format similar to the previous one, but simpler

Character	Contents	Meaning
-----	-----	-----

0-1	CR-LF	carriage return/line feed to separate the blocks on a listing as in the TDL Standard.
2	:	colon to indicate the start of an absolute block on the object tape.
3-4	hex digits	as in the TDL Standard, this byte contains the count of program/data bytes following in this block.
5-8	hex digits	as in the TDL Standard, these two bytes specify the load address for the block.
9-10	hex digits	this byte is ignored.
11-(n-1)	hex digits	these bytes contain the program/data to be loaded. There are no relocation control bytes because the data is absolute only.
n-(n+1)	hex digits	this byte contains the checksum which is formed in the same manner as in the TDL Standard.